

★ Member-only story

Step-by-Step Guide to Setting Up Microservices in Node.js with Docker & Kubernetes (2025)



Somendradev

Following ▾

4 min read · 5 hours ago



1



Microservices aren't just a buzzword anymore — they're a **game-changer** for building scalable apps. Instead of stuffing all your logic into a giant monolith, you break it down into **small, independent services** that are easier to scale, deploy, and maintain.

NODE.JS ARCHITECTURE PATTERNS

FOR SCALABLE APPS





This Image Generated By AI.

And when you pair **Node.js** with **Docker** and **Kubernetes**, you get a powerhouse stack to handle millions of users without breaking a sweat.

Let's break this down **step-by-step** with clear examples.

◆ **Why Microservices? (In Simple Words)**

Think of a restaurant:



- 
- A **monolithic kitchen** is where the same chef cooks *everything* — pizza, sushi, burgers. If one station slows down, everyone suffers.
 - A **microservices kitchen** has **specialized chefs** (pizza chef, sushi chef, burger chef) working independently. If one chef is overloaded, you can hire more just for that role.

In software terms:

- **Monolith:** One big Node.js app doing everything.
- **Microservices:** Multiple Node.js services, each responsible for one thing (auth, payments, orders, etc.).

◆ Tech Stack Overview

- **Node.js:** Fast, lightweight runtime for building APIs.
 - **Docker:** Packages each service with its dependencies into containers.
 - **Kubernetes (K8s):** Orchestrates and scales containers automatically.
- 

🔗 Step 1: Design Your Microservices

Let's say we're building a Food Delivery App. Break it down:

ServiceResponsibilityAuth ServiceHandle login/signupOrder ServiceManage
ordersPayment ServiceHandle paymentsNotification ServiceSend emails,
SMS

Each service is its own Node.js app, with its own database.

🔗 Step 2: Build a Simple Node.js Microservice

Let's create a simple Order Service:

📁 order-service/index.js

```
const express = require("express");
const app = express();
app.use(express.json());

app.get("/orders", (req, res) => {
```

```
res.json([ { id: 1, item: "Pizza", status: "Delivered" } ]));
});

app.post("/orders", (req, res) => {
  const { item } = req.body;
  res.status(201).json({ message: `Order placed for ${item}` });
});

app.listen(3000, () => {
  console.log("Order Service running on port 3000");
});
```

Run it:

```
node index.js
```

Visit <http://localhost:3000/orders> → You'll see sample orders.

Node.js Architecture Patterns for Scalable Apps (2025 Guide)

Node.js has become the go-to choice for building high-performance, scalable apps — especially real-time systems, APIs...

medium.com



🔧 Step 3: Dockerize the Service

Create a Dockerfile :

```
# Use Node.js base image
FROM node:18

# Create app directory
WORKDIR /usr/src/app

# Copy files
COPY package*.json ./
RUN npm install
COPY . .

# Expose port & start app
EXPOSE 3000
CMD ["node", "index.js"]
```

Build and run:

```
docker build -t order-service .  
docker run -p 3000:3000 order-service
```

Now your service is running **inside a container**.

✂ Step 4: Create Multiple Services

Repeat the same for **Auth Service**, **Payment Service**, etc.
Each service should have its own Docker image.

✂ Step 5: Set Up Docker Compose for Local Development

docker-compose.yml :

```
version: '3'  
services:  
  order-service:  
    build: ./order-service
```



```
ports:
  - "3000:3000"
auth-service:
  build: ./auth-service
  ports:
    - "3001:3000"
```

Run all services:

```
docker-compose up --build
```

Now you have multiple services running locally, each in its own container.

How to Scale MySQL Databases for High-Traffic Node.js Apps

If you've built a Node.js app that's starting to attract thousands — or even millions — of users, congratulations! 🎉...

[medium.com](#)

🔧 Step 6: Deploy to Kubernetes

Once services are containerized, let's put them in K8s.

order-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-service
spec:
  replicas: 3
  selector:
    matchLabels:
      app: order-service
  template:
    metadata:
      labels:
        app: order-service
    spec:
      containers:
        - name: order-service
          image: order-service:latest
          ports:
            - containerPort: 3000
---
apiVersion: v1
kind: Service
metadata:
  name: order-service
```

```
spec:
  selector:
    app: order-service
  ports:
    - protocol: TCP
      port: 80
      targetPort: 3000
```

Deploy it:

```
kubectl apply -f order-deployment.yaml
```

Kubernetes now runs **3 replicas** of your Order Service, auto-managing scaling and health checks.

🔗 Step 7: Add an API Gateway

Microservices can get messy without a **single entry point**. Use:

- 
- **NGINX** or **Kong API Gateway** to route requests:
 - `/auth` → Auth Service
 - `/orders` → Order Service
 - `/payments` → Payment Service

This makes your system feel like **one app** to users.

🔧 **Step 8: Monitoring and Scaling**

- Use **Prometheus + Grafana** to monitor container health.
- Use **Horizontal Pod Autoscaler (HPA)** in Kubernetes to auto-scale pods based on traffic.
- Add **Redis** or **RabbitMQ** for async communication between services.

Key Benefits of This Setup



- 
- ✓ **Scalability:** Scale only the services under heavy load (e.g., Payment Service).
 - ✓ **Fault Tolerance:** If one service crashes, others still work.
 - ✓ **Independent Deployments:** Push updates without touching the entire system.
 - ✓ **Flexibility:** Use different tech stacks per service if needed.

Real-World Example

Imagine Uber Eats:

- Your **Order Service** scales massively at dinner time.
- **Auth Service** traffic stays low.
- With Kubernetes, only Order Service scales up → **lower costs & faster response times.**

TL;DR



Microservices + Docker + Kubernetes is like giving your Node.js app a **superpower suit**:

- Build small, independent services.
- Package each in Docker containers.
- Use Kubernetes to scale automatically.
- Add an API Gateway and monitoring for production readiness.

Automate Document Processing in Node.js Using AI OCR & NLP

Imagine this: you run a business that receives hundreds — sometimes thousands — of documents every day. They come in...

blog.gopenai.com

Microservices

Nodejs

Docker

Kubernetes

Nodejs Microservices

